Development Plan

Pacca Out of School Club

Author: Leroy Russell

Website url http://localhost/server/pacca/index.php

Content

Introduction 2
System Requirements 2
Functional requirements 2
Non-functional requirements 2
Actors and use case 3
Admin use case list 3
Staff use case list 4
Member use case list 4
Use case diagram 4
Use case descriptions 5
Activity Modelling 6
U3. Add Staff 6
Class Entities 7
Entity classes 7
Attributes 8
Association 8
Class Diagram 9
Prototype 9
Site Map 9
User Interfaces 10
Implementation 12
Front-end 12
Back-end 13
Testing 13
Compliance 13
Accessibility 13
Browser Compatibility 13
Enhancements 14

Introduction

This report details the design, implementation and testing of the website developed for PACCA Out of School Summer Club.

PACCA provide a variety of childcare services including but not limited to taking child to and from school and providing children with breakfast and evening tea. School breaks and summer holiday the club provide a childcare service that includes activities and day trips. Data processing for the current system is outdated, resulting in time consuming administrative duties.

The benefits of transferring from the current system include the ability to reach a greater audience, growth in the productivity, improve the data processing currently used by PACCA and increase potential for company growth and expansion. A web and mobile application will provide PACCA with a much-needed centralised information system.

The web application will be an information system for the services provided by PACCA, as the internet has a greater potential to reach the targeted audience more effectively, in addition to reaching a new audience.

System Requirements

The following functional and non-functional requirements will used for the system.

Functional requirements

1. Navigation and interactive elements must produce consistent results.
2. Allow *User* to register for membership.
3. System must store *Member* information i database.
4. Allow *Member* to cancel membership.
5. Allow *Administrator*, *staff* and *Member* to log in to the system.
6. Provide managerial and administrative tools to *Administrator* and *Staff*.
7. Display appropriate message for system errors

Non-functional requirements

1. User interfaces must be aesthetically consistent.
2. System must conform to W3C standards
3. System must be accessible and useable
4. System must be fully responsive for desktop and mobile devices.
5. System must provide support for user-agents.

Actors

The actors required to interact with the system.

Actors required

1. User
2. Member
3. Database

Actors description

1. *Administrator:* A person who is responsible for entire company
2. *Staff:* A person employed to by company to provide childcare
3. *Member:* A person who registers their details to be stored by Pacca
4. *Database:* An organised mechanism for storing, managing and retrieving information

Use cases

The uses cases required to achieve the systems functionality.

Administrator use case list

U1. *AddBudget*: *Add budget to system.*
U2. *EditBudget*: Edit budget details and update system.
U3. *DeleteBudget:* Delete budget from system.
U4. *AddStaff: Add staff to the* system.
U5. *EditStaff: Edit staff details and update* system.
U6. *DeleteStaff*: Delete staff from the system.
U7. *DeleteMember*: Delete member from system.
U8. *AddActivity: Add activity to* system.
U9. *EditActivity: Edit activity details and update* system.
U10. *DeleteActivity*: Delete activity from system.
U11. *AddEvent: Add event to* system.
U12. *EditEvent: Edit event details and update* system.
U13. *DeleteEvent*: Delete event from system.
U14. *ViewEntity*: View information stored related to entity.
U15. *Login*: Log into the system.
U16. *Logout:* Log out of the system.

Staff use case list

U1. *ViewActivity:* View activity from database.
U2. *ViewEvent*: View event from database.
U3. *ViewStaff*: View staff from database.
U4. *ViewMember*: View members from database.
U5. *ViewChild*: View children from database.
U6. *AssignStaff:* Assign staff to activity / event and update database.
U7. *ReassignStaff*: Reassign staff to or remove from activity / event.
U8. *Login*: Log into the system.
U9. *Logout:* Log out of the system.

Member use case list

U1. *AddChild:* Add child to database.
U2. *EditChild:* Edit child details and update database.
U3. *DeleteChild*: Delete child from the database.
U4. *AddActivity*: Add activity for child participation.
U5. *DeleteActivity*: Delete from child participation.
U6. *AddEvent*: Add event for child participation.
U7. *Deletevent:* Delete from child participation.
U8. *PayEvent*: Pay for event provided by PACCA.
U9. *ViewEntity*: View information stored related to entity.
U10. *Login*: Log into the system.
U11. *Logout:* Log out of the system.
U12. *EditMembership*: CaEditncel membership and update database.
U13. *DeleteMembership*: Remove membership data from database.

Use case diagram

Figure 1 shows the extended relationships from the Login use case for *Administrator*.

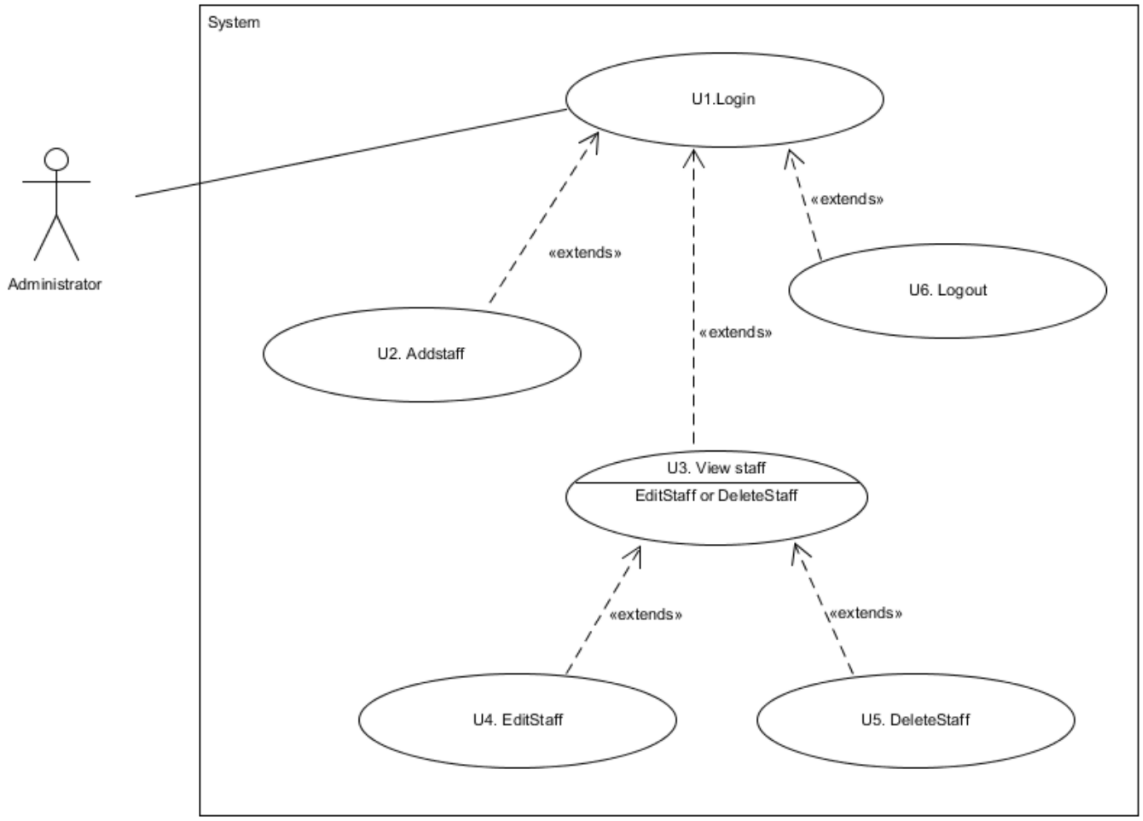

Figure 1. The use case and extended relationships from Login.

Use case descriptions

The main flow for the use cases are detailed in the use case descriptions.

U1 - Login	
Brief Description	This use case allows Staff to login.
Actors	*Admin, Staff, Member.*
Preconditions	Actors must have *username* and *password* details.
Main Flow	1. *Actor* selects login from website. 2. *System* displays login form on page 3. *Actor* must enter all the mandatory fields to complete *Login*. 4. If Login complete, system validates and verifies Actor 5. If valid and verified Actor can interact with system 6. Else display error message.
Alternatives Flow	7. *Admin can use* a relational database management system. *Staff* and *Member* must contact *Admin*
Post-conditions	Access is granted to account based on actor.

Table 1: The use case description for *Login.*

U2 - AddStaff	
Brief Description	This use case allows *Admin* to add *Staff*.
Actors	*Administrator.*
Preconditions	*Admin must be logged in.*
Main Flow	1. *Admin* selects add staff function. 2. *System displays add staff page* 3. Admin must enter all the mandatory form fields. 4. If fields complete, system validates and searches for staff 5. If staff is found, system is not changed. 6. If not found system stores data from form field to system 7. If data stored, Administrator is able view, edit and delete Staff.
Alternatives Flow	1. Use a relational database management system
Post-conditions	Actor can edit or delete any selected Staff.

Table 2: Use case description for *AddStaff*.

Activity Modelling

Activity modelling captures the dynamic behaviour of the system and show the flow from action to another.

The activity diagram shown in figure 2, illustrates the flow of action for U3 add staff.

U2. Add Staff

1. Admin select add staff.
2. *System displays add staff page*
3. Admin must enter all the mandatory form fields.
4. If fields complete, system validates and searches for staff
5. If staff is found, system is not changed.
6. If not found system stores data from form field to system
7. If data stored, Administrator is able view, edit and delete Staff.

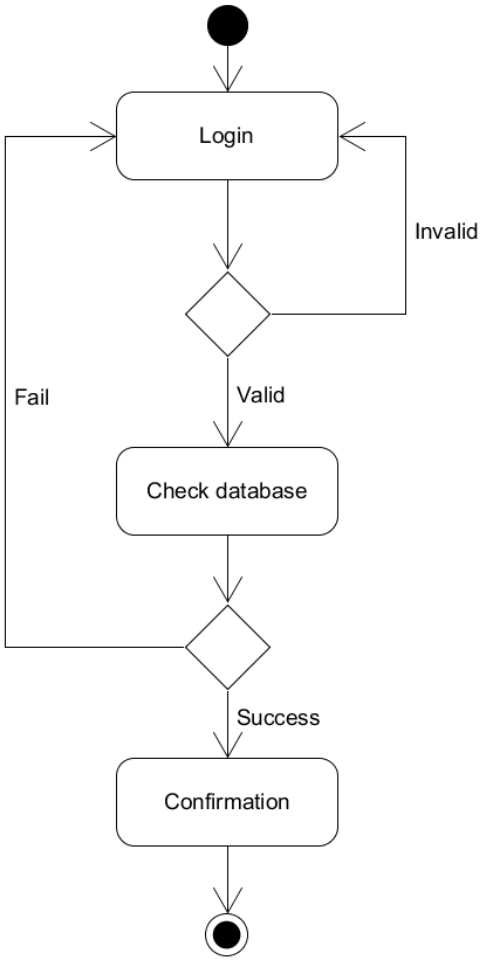

Figure 2. Activity diagram for adding staff to system.

Class Entities

The structure of the system is defined by the entities and their relationship.

Entity classes

The class entities provide the foundation for system interaction. Figure 3 shows the entities required for the system.

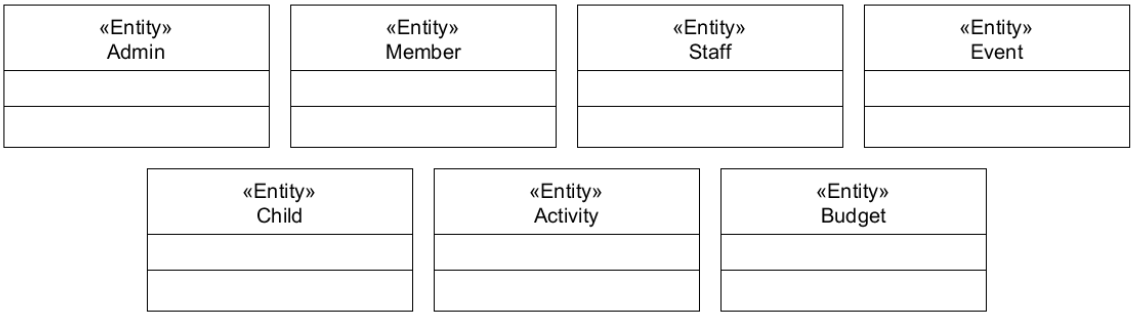

Figure 3. Class entities used for the system.

Attributes

The *attributes* for the entities define the characteristics and the data that to be stored from each instance. Figure 4 shows the attributes for each class.

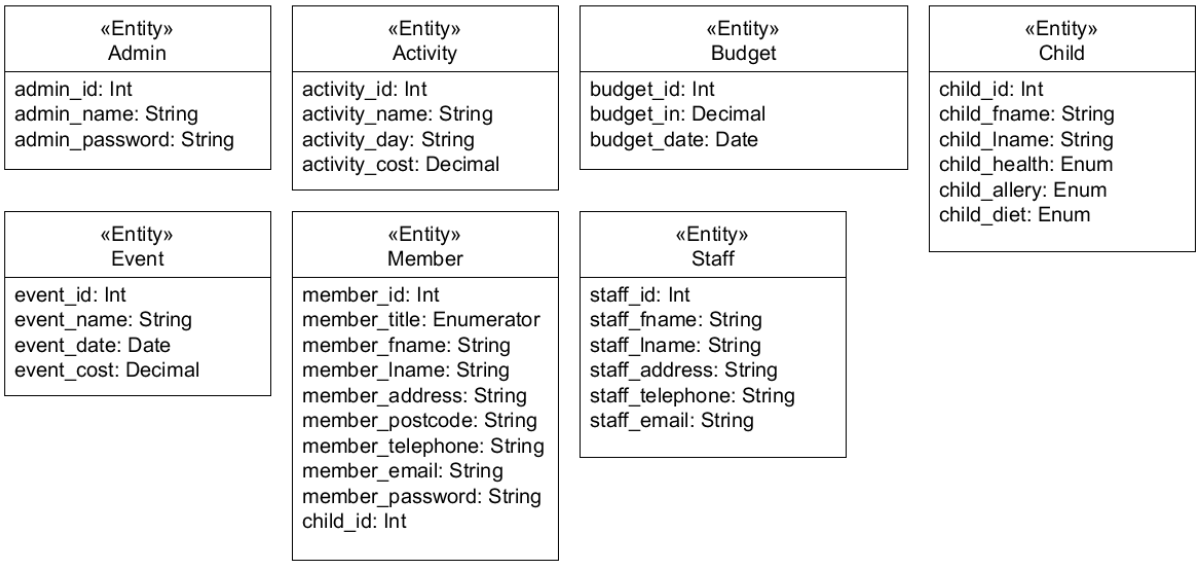

Figure 4. The attributes for the entities of the system.

Association

The relationship between each entity is shown in Figure 5.

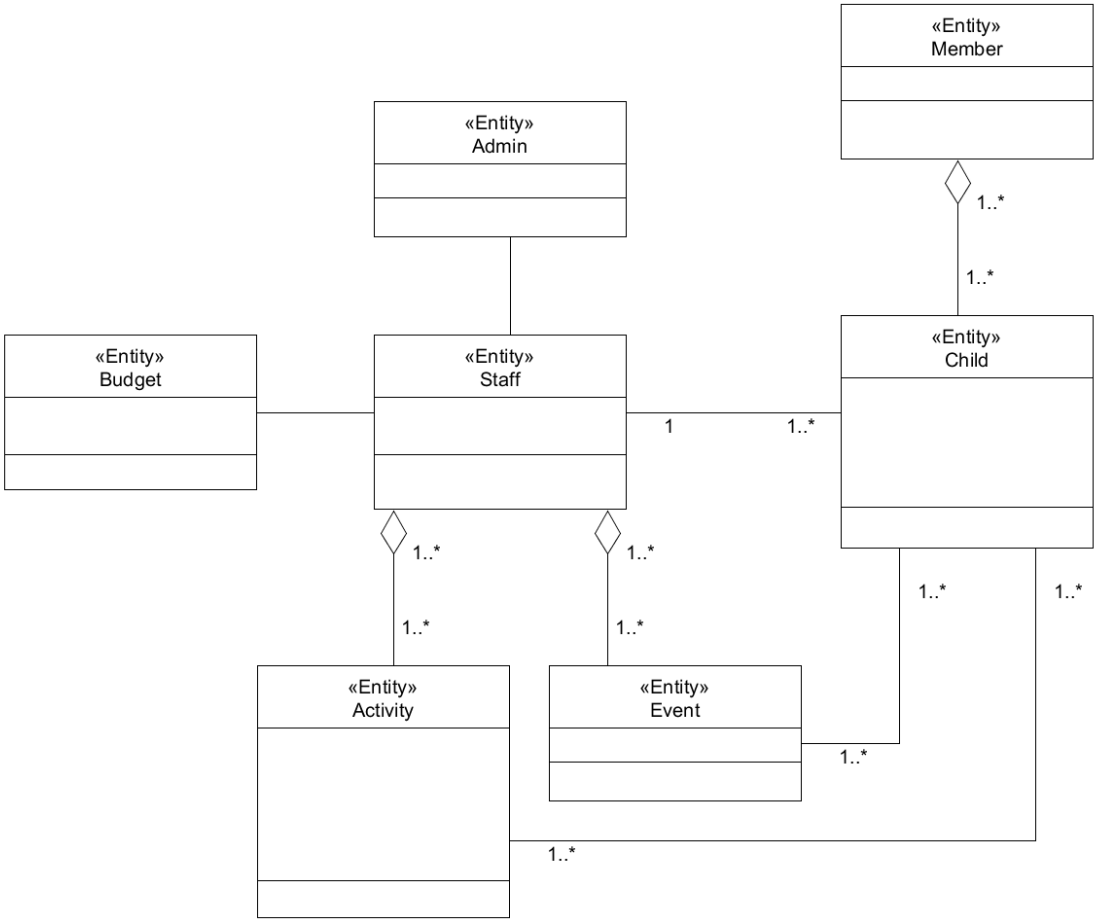

Figure 5. Association relationship for the entities of the system.

Class Diagram

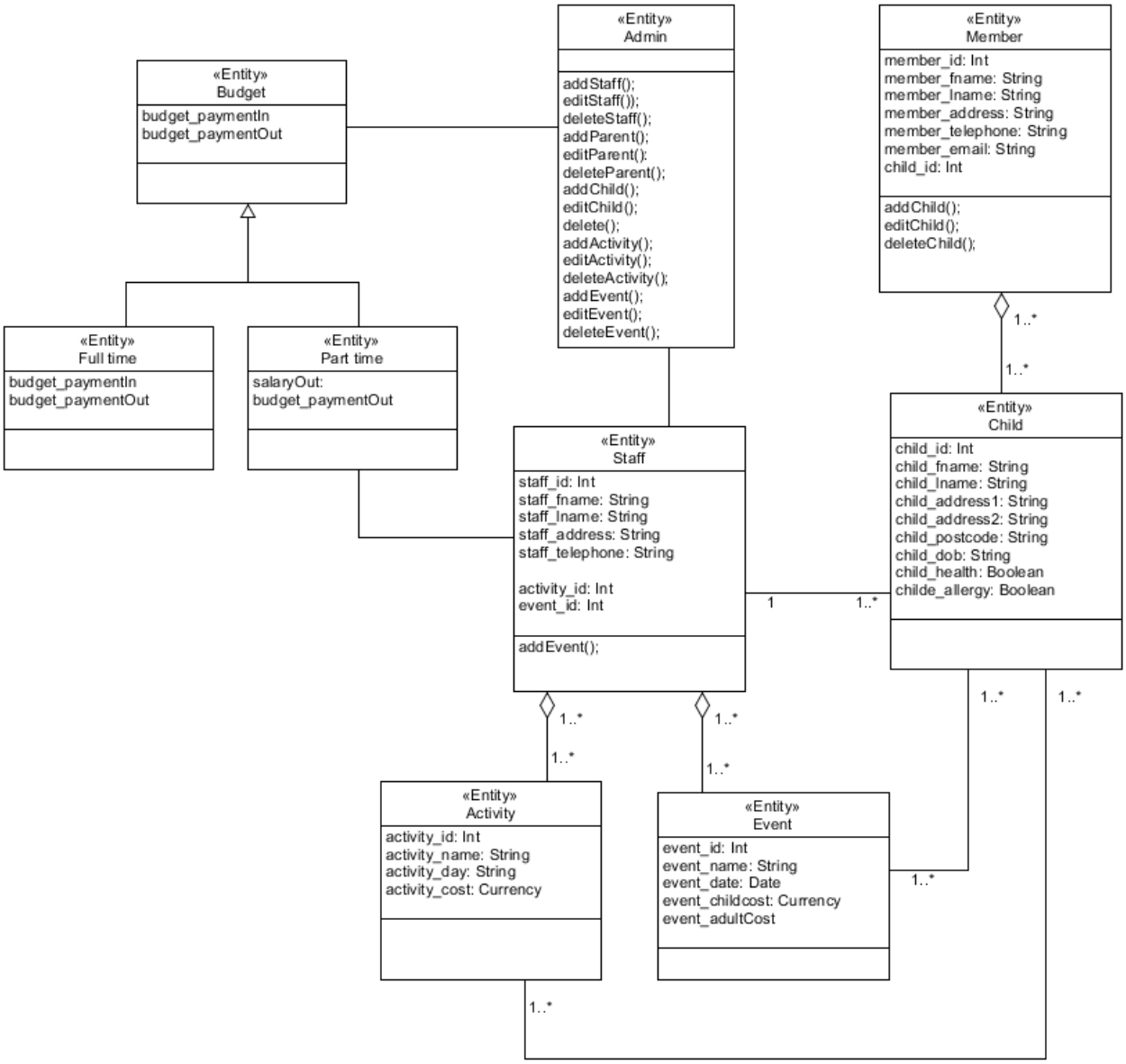

Figure 6. Class diagram showing the relationship between entities of the system.

Prototyping

Prototyping demonstrates the functionality of the system.

Site Map

The site map shown in figure 7 represents the structure of the website.

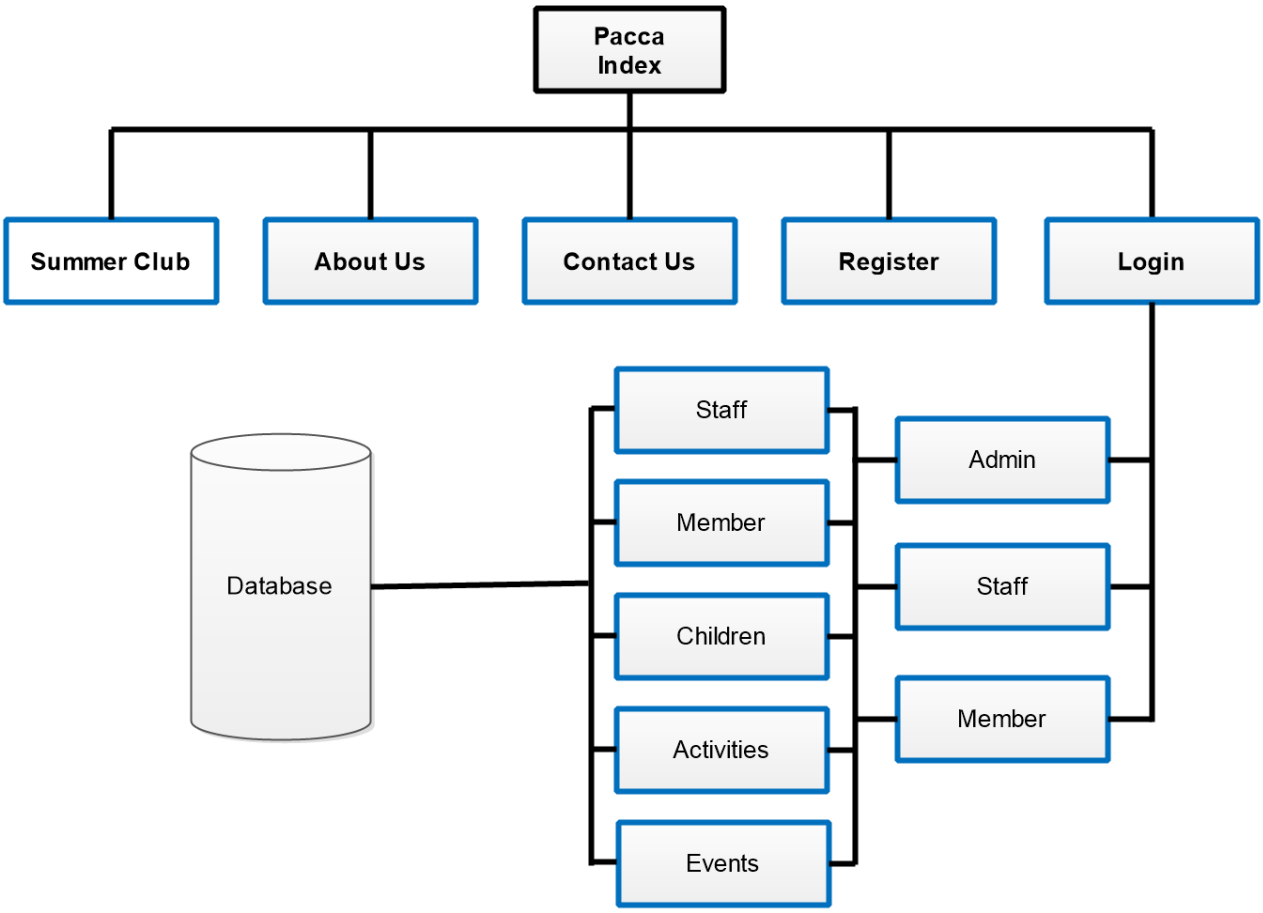

Figure 7. The site map for the website.

User Interfaces

The graphical user interface for the website provides the necessary textual and visual information to assist in achieving the desired functionality

Landing/Index Page

The landing page must display information about the services provided by the company, including testimonials from existing customers and contact details.

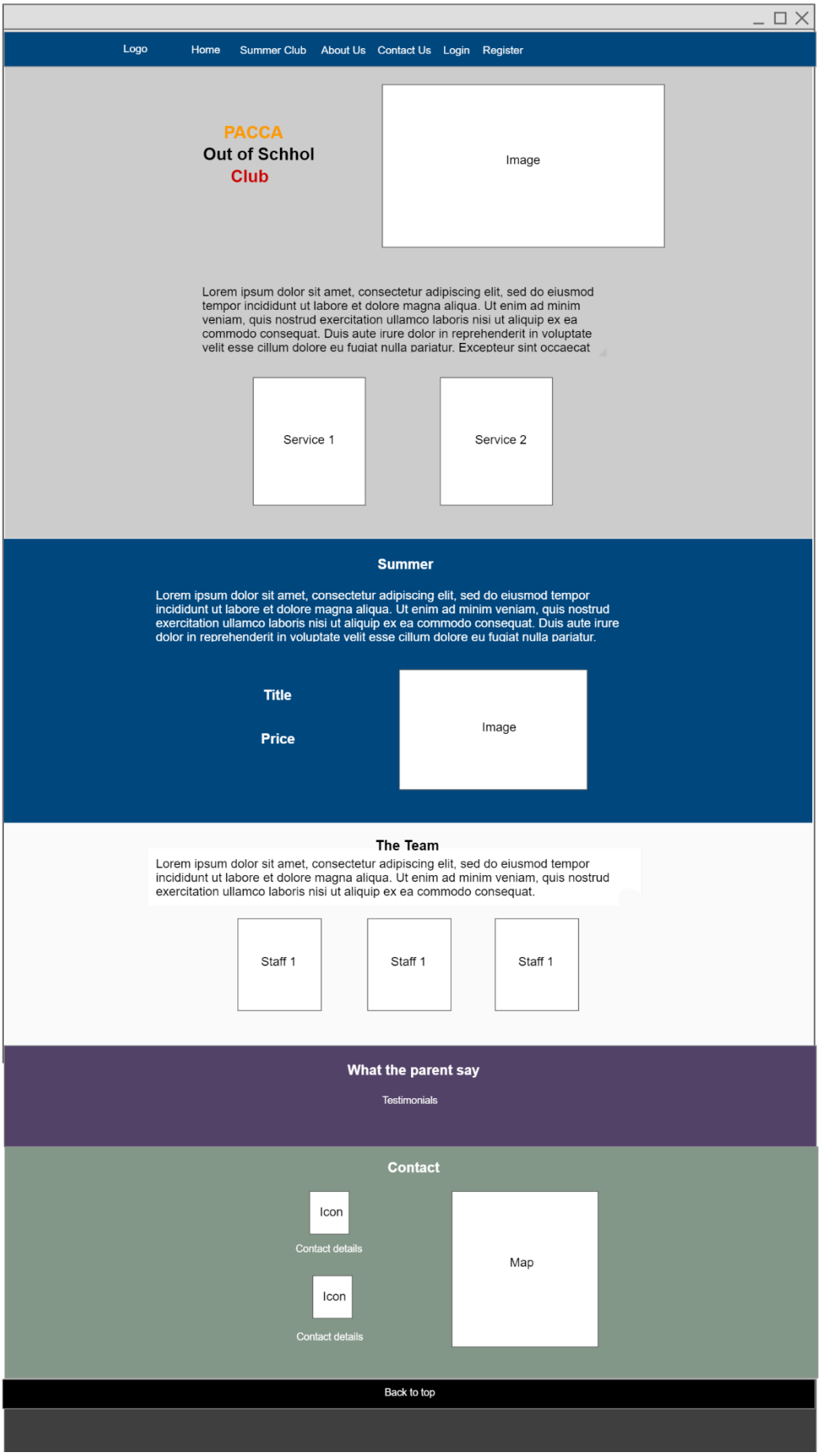

Figure 8. The Landing page for the website.

Login Page

The Login page must display a form which allows a user to log in to the system.

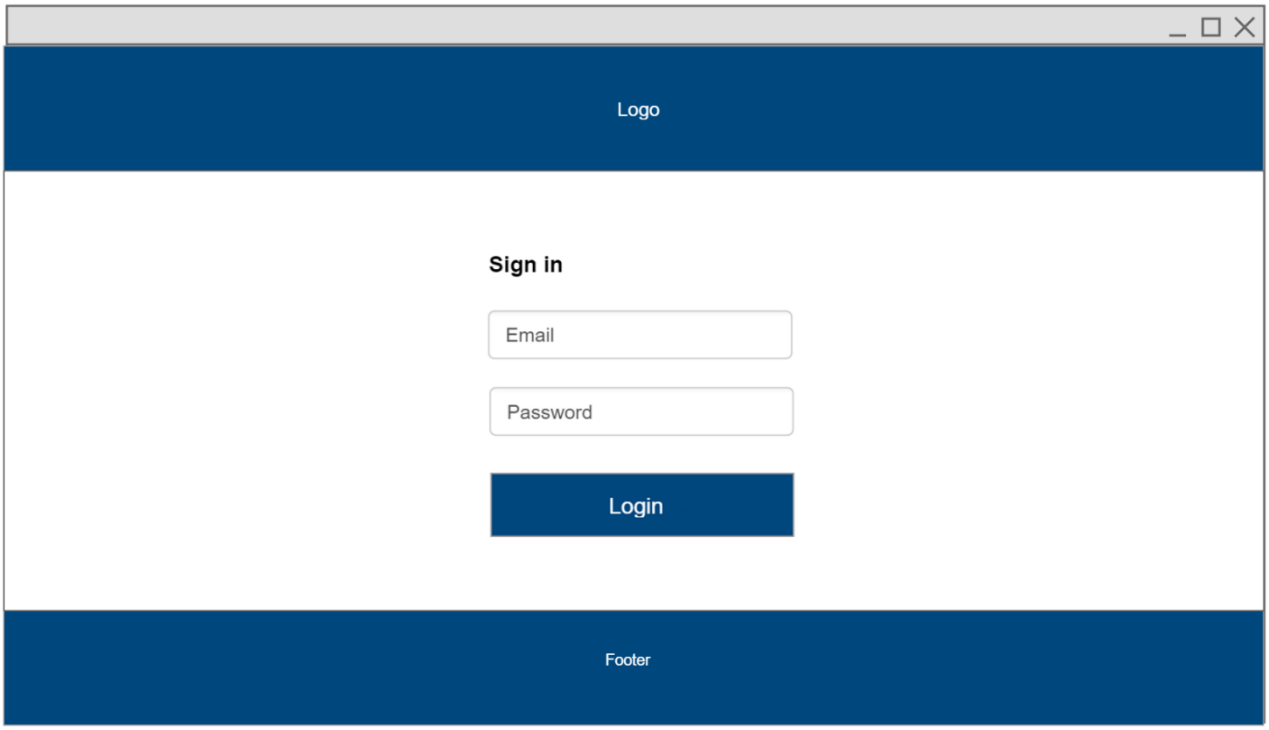

Figure 10. The Login page for the website.

Implementation

The following technologies will be used to build the structure, presentation and functionality of the system.

Front-end

1. HTML5
2. CSS3 and W3.CSS
3. JavaScript
4. Laravel
5. Krita

Technology	Usage
HTML5	Structure and layout of the page elements
CSS3 & W3.CSS	Styling and responsiveness of the web pages
jQuery	Functionality for the website
Laravel	Framework used to achieve front-end development
Krita	Computer software used for images manipulation and creation

Table 3. Technologies used for the front-end development.

Back-end

1. Laravel
2. MySQL

Technology	Usage
Laravel	Framework used to communicate with database
MySQL	The database used for data storage and retrieval

Table 4. Technologies used for the back-end development.

Note: The back-end development has not been implemented!

Testing

The following tests ensure the website meets web standards, browser compatibility and accessibility.

Compliance

HTML5 and CSS3 have been validated and comply with W3C standards.

Accessibility

ARIA landmark roles, ARIA properties and skip links have been implemented to enhance website accessibility and usability.

Browser Compatibility

All web browsers tested are responsiveness and consistent results across all browsers. The results of the tests are shown in Table 5.

Web Browser	Version	Result

Firefox	79	Pass
Chrome	84	Pass
Microsoft Edge	44	Pass
Internet Explorer	11	Pass
Opera	69	Pass
Safari	5.3	Fail Issue: Alignment and displaying of elements

Table 5. The test results for web browser compatibility.

Enhancements

Further improvements for the website are listed in table 6.

Task	Improvement
Interfaces	• Rearrange the layout of the activities and events
Implementation	• Use back-end to
Browser Compatibility	• HTML5 shiv and other techniques should be used to make website compatible with older browsers

Table 6. Test results for web browser compatibility.